

Credit Card Approval Prediction

Project Description:

Credit Card Approval Prediction harnesses the power of **Artificial Intelligence and Machine Learning** to automate the credit card approval process by accurately predicting whether an applicant is eligible for approval or rejection. The platform includes an **Applicant Information Module**, where users enter personal and financial details such as age, gender, income, debt, employment status, credit score, marital status, industry, ethnicity, and citizenship. The system preprocesses the input data using **Label Encoding** and **StandardScaler**, ensuring it is suitable for machine learning prediction.

Utilizing the **Random Forest Classifier** from the **Scikit-learn** library, the application analyzes the applicant's information and generates an instant prediction with a confidence score. Developed using **Python, Flask, HTML, and CSS**, the web application provides a simple and user-friendly interface for entering applicant details and viewing prediction results. The trained model and preprocessing components are securely stored using **Pickle**, enabling fast and consistent predictions. This project helps financial institutions reduce manual effort, improve decision-making accuracy, and speed up the credit card approval process by delivering reliable approval or rejection predictions in real time.

Scenarios

Scenario 1: Eligible Applicant

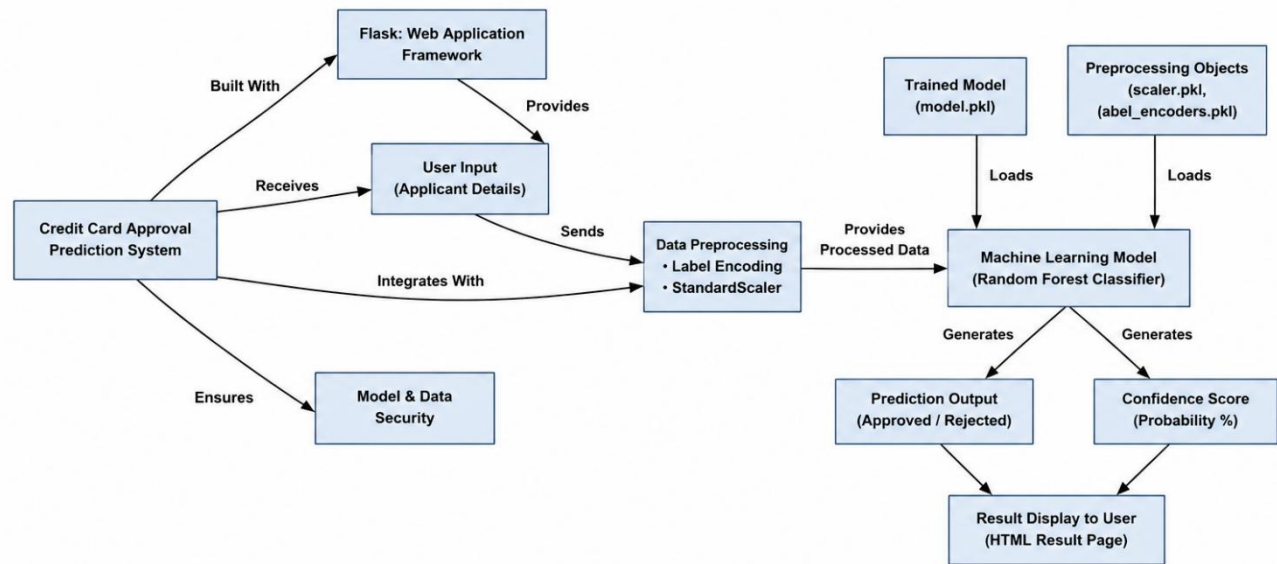
A user enters applicant details such as a good credit score, stable employment, high income, low debt, and no prior defaults through the web application. The system preprocesses the input data, applies the trained Random Forest model, and predicts the application as **APPROVED** along with a confidence score, enabling quick and reliable decision-making.

Scenario 2: High-Risk Applicant

A user submits an application with a low credit score, high debt, unstable employment, and a history of prior defaults. The system processes the applicant's information using the trained machine learning model and predicts the application as **REJECTED**, helping financial institutions reduce the risk of approving unreliable applicants.

Scenario 3: A bank employee or applicant enters the required personal and financial details through the Flask-based web application. The system validates the input, performs data preprocessing, generates a prediction using the Random Forest Classifier, and instantly displays the **APPROVED** or **REJECTED** result along with the prediction confidence, making the credit card approval process faster, more consistent, and user-friendly.

► Technical Architecture:



Description: The Credit Card Approval Prediction system follows a modular architecture to deliver accurate predictions. The Flask web application collects applicant details through the user interface. The input data is preprocessed using Label Encoding and StandardScaler to convert and normalize features. The processed data is then passed to the Random Forest Classifier model, which loads the trained model and preprocessing objects to generate the prediction result and confidence score. Finally, the result is displayed to the user on the result page.

Pre-requisites:

- **Flask Documentation:** <https://flask.palletsprojects.com/>
- **Scikit-learn Documentation:** <https://scikit-learn.org/>
- **Pandas Documentation:** <https://pandas.pydata.org/docs/>
- **NumPy Documentation:** <https://numpy.org/doc/>
- **Python Documentation:** <https://docs.python.org/3/>
- **HTML & CSS Tutorials:** <https://www.w3schools.com/>
- **Git Documentation (Optional):** <https://git-scm.com/doc>

Project Workflow:

Milestone 1: Data Preparation and Model Development

- Activity 1.1: Collect and preprocess the credit card approval dataset by handling missing values and encoding categorical features.
 - Activity 1.2: Split the dataset into training and testing sets using Scikit-learn.
 - Activity 1.3: Apply feature scaling using StandardScaler to normalize numerical data.
 - Activity 1.4: Train the Random Forest Classifier and evaluate its performance using accuracy and classification metrics.
-

Milestone 2: Machine Learning Model Integration

- Activity 2.1: Save the trained machine learning model, scaler, and label encoders using the Pickle library.
 - Activity 2.2: Integrate the saved model into the Flask application for real-time credit card approval prediction.
-

Milestone 3: Backend Development (app.py)

- Activity 3.1: Develop the Flask backend to handle user requests and manage application routing.
 - Activity 3.2: Process applicant details, perform preprocessing, generate predictions, and display approval or rejection results with confidence scores.
-

Milestone 4: Frontend Development

- Activity 4.1: Design a responsive user interface using HTML and CSS for collecting applicant details.
 - Activity 4.2: Develop dynamic result pages using Flask's render_template() to display prediction outcomes (Approved/Rejected) along with confidence percentages.
-

Milestone 5: Testing and Deployment

- Activity 5.1: Test the application using various applicant inputs to verify prediction accuracy and system reliability.
 - Activity 5.2: Run the application on the local Flask development server and ensure all required Python dependencies are installed correctly.
-

Milestone 6: Conclusion

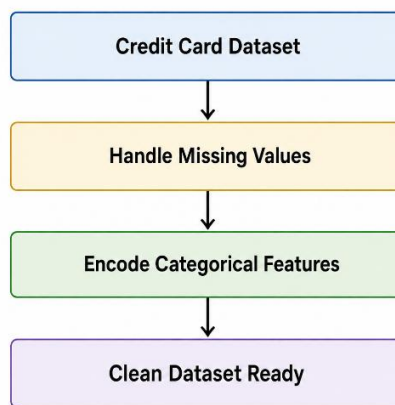
- Successfully developed an AI-based Credit Card Approval Prediction System using Python, Flask, and Random Forest Classifier.
- The application accurately predicts whether a credit card application is likely to be Approved or Rejected based on applicant information.
- The system provides a simple, user-friendly web interface with fast predictions, helping automate the credit approval decision-making process.

Milestone 1: Data Preparation and Model Development

Activity 1.1: Collect and Preprocess the Dataset

The credit card approval dataset was collected and imported into the Python environment using the Pandas library. During preprocessing, missing values were handled to improve data quality and ensure consistency. Categorical attributes such as Industry, Ethnicity, and Citizen were converted into numerical values using the Label Encoder technique. This preprocessing step prepared the dataset for machine learning model training.

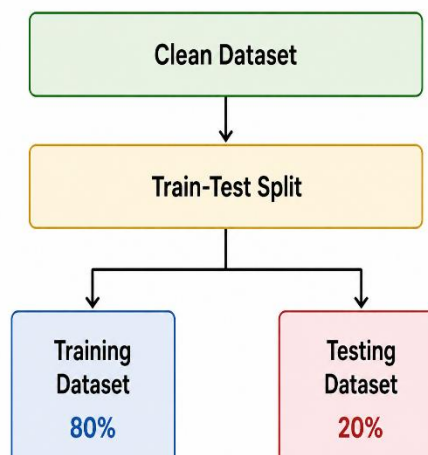
1.1: Data Collection and Preprocessing



Activity 1.2: Split the Dataset into Training and Testing Sets

After preprocessing, the cleaned dataset was divided into training and testing datasets using the `train_test_split()` function from the Scikit-learn library. Eighty percent of the data was used to train the machine learning model, while the remaining twenty percent was reserved for testing and evaluating the model's performance on unseen data.

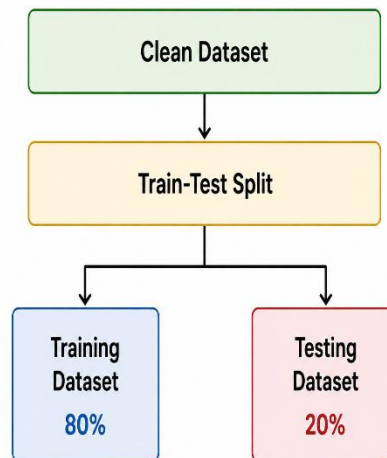
1.2: Dataset Splitting Process



Activity 1.3: Apply Feature Scaling

Feature scaling was performed using Standard Scaler to normalize numerical attributes. This process transforms the numerical values so that they have a mean of zero and a standard deviation of one. Scaling improves model performance by ensuring that all numerical features contribute equally during training and prediction.

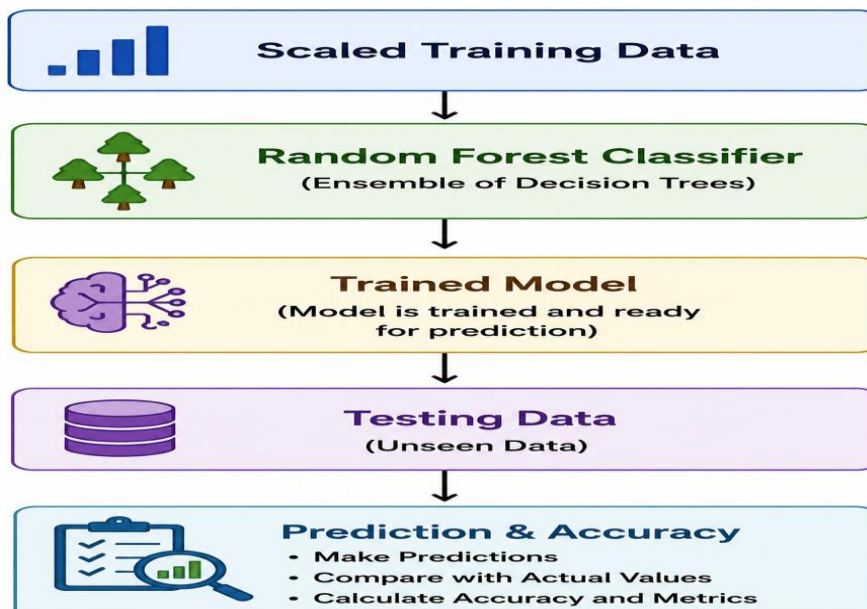
1.3: Feature Scaling using Standard Scaler



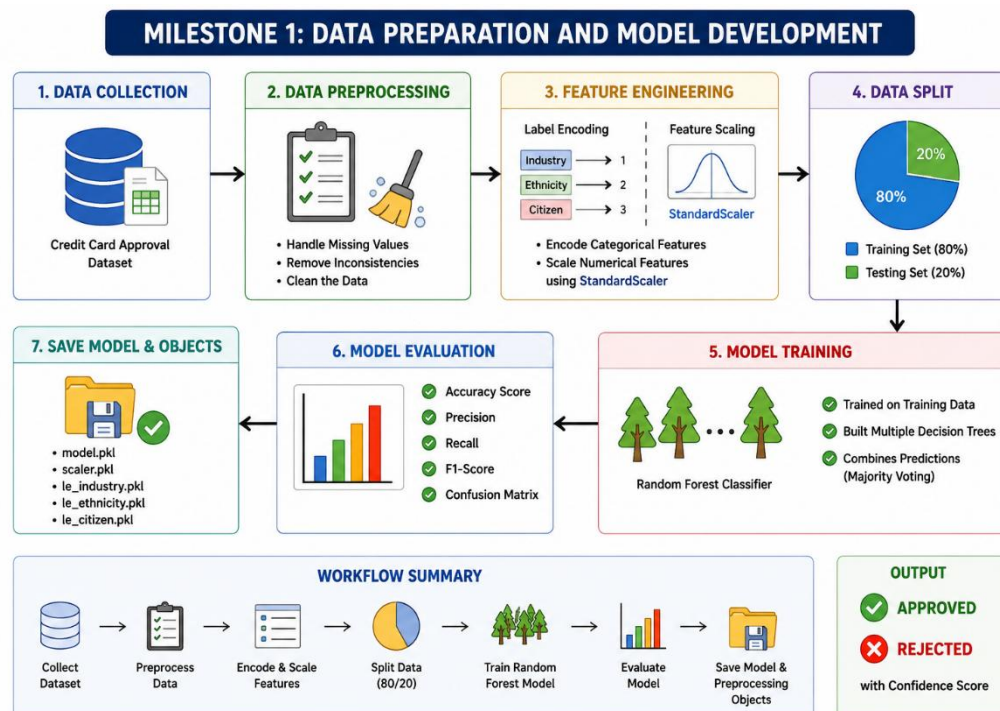
Activity 1.4: Train and Evaluate the Random Forest Model

The preprocessed and scaled training data was used to train the Random Forest Classifier. After training, the model was evaluated using the testing dataset. Performance was measured using accuracy score and classification metrics, which helped determine the model's ability to correctly predict whether a credit card application should be Approved or Rejected. The trained model and preprocessing objects were then saved as pickle files for integration with the Flask web application.

1.4: Random Forest Model Training



In this milestone, the Credit Card Approval Prediction system is developed by preparing the dataset and building a machine learning model capable of predicting whether a credit card application will be approved or rejected. The raw dataset is first cleaned by handling missing values and converting categorical attributes into numerical values using label encoding. The processed dataset is then divided into training and testing sets to ensure unbiased model evaluation. Numerical features are standardized using **Standard Scaler** to improve the performance and consistency of the machine learning algorithm. Finally, a **Random Forest Classifier** is trained on the processed data and evaluated using accuracy and classification metrics. The trained model, along with the scaler and label encoders, is saved for use in the Flask web application, enabling fast and reliable credit card approval predictions.



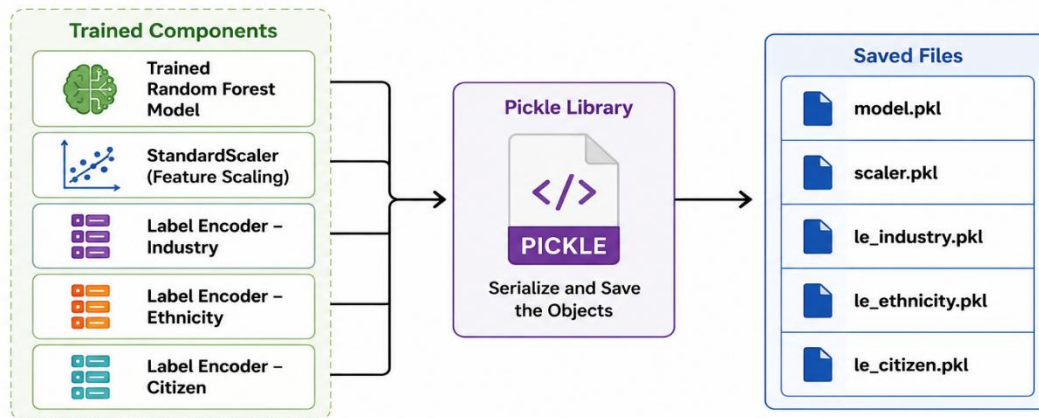
Milestone 2: Machine Learning Model Integration

In this milestone, the trained machine learning model is integrated with the Flask web application to perform real-time credit card approval predictions. The trained **Random Forest Classifier**, **Standard Scaler**, and **Label Encoders** are saved using the Pickle library so they can be reused without retraining the model. These saved files are then loaded into the Flask backend, where user input is preprocessed, scaled, and passed to the model for prediction. The application displays the final approval or rejection result along with the confidence score, providing users with a fast, accurate, and user-friendly prediction system.

Activity 2.1: Save the Trained Model and Preprocessing Files

Activity 2.1: Save the Trained Model and Preprocessing Files

The trained machine learning model, scaler, and label encoders are saved using the Pickle library for later use in the Flask application.



Purpose

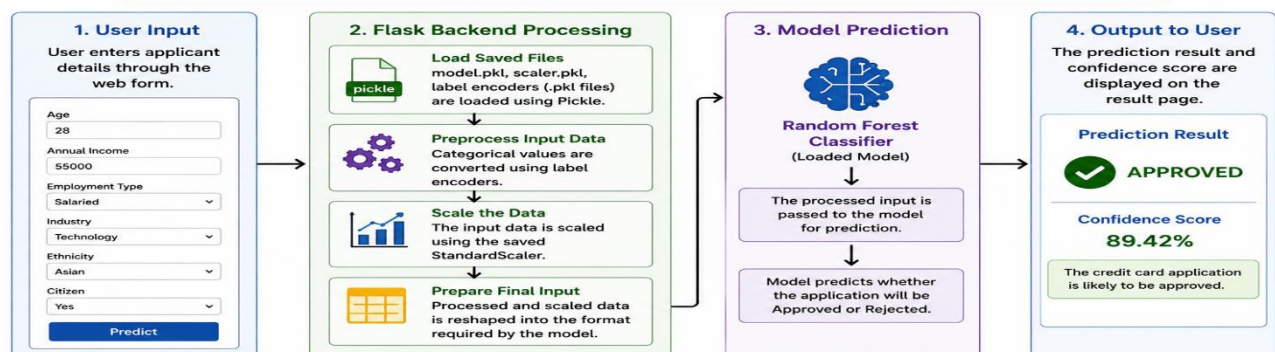
These saved files are loaded in the Flask application to preprocess the input data and make real-time predictions without retraining the model.

In this activity, the trained Random Forest model, Standard Scaler, and Label Encoders are saved using the Pickle library. Saving these files allows the application to reuse the trained model without training it every time the application starts. The saved files include model.pkl, scaler.pkl, le_industry.pkl, le_ethnicity.pkl, and le_citizen.pkl, which are later loaded by the Flask application for prediction.

Activity 2.2: Integrate the Model into the Flask Application

Activity 2.2: Integrate the Saved Model into the Flask Application

The saved machine learning model, scaler, and label encoders are loaded in the Flask application to provide real-time credit card approval prediction.



Summary

The Flask application loads the saved model and preprocessing files, processes the user input, and returns the prediction result in real-time with confidence score.

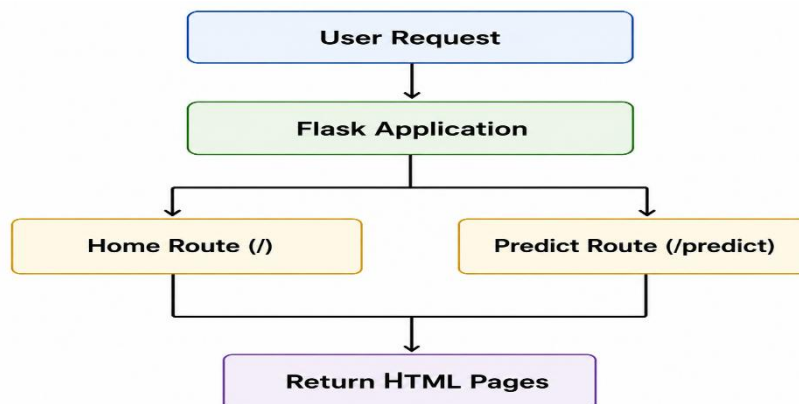
In this activity, the saved machine learning model and preprocessing files are loaded into the Flask application using the Pickle library. When a user enters applicant details through the web interface, the Flask backend encodes categorical values, scales the input using the saved scaler, and sends the processed data to the Random Forest model. The model predicts whether the credit card application is Approved or Rejected and displays the prediction along with the confidence percentage on the result page, enabling real-time credit card approval prediction.

This milestone focuses on connecting the trained machine learning model with the web application to enable real-time credit card approval prediction. The integration ensures that every applicant's information entered through the user interface is processed automatically by the backend, allowing the prediction model to generate accurate results instantly. The Flask application acts as the bridge between the user interface and the machine learning model, providing a seamless workflow from data input to prediction output. This integration makes the system efficient, reliable, and easy to use, allowing users to obtain credit card approval decisions along with confidence scores through a simple web-based interface.

Milestone 3: Backend Development (app.py)

In this milestone, the Flask backend is developed to manage the complete workflow of the Credit Card Approval Prediction system. The backend receives applicant details submitted through the web interface, processes the input data, communicates with the trained machine learning model, and returns the prediction result. It also manages application routing, handles user requests efficiently, and displays the final approval or rejection decision with the corresponding confidence score, ensuring smooth interaction between the user interface and the prediction model.

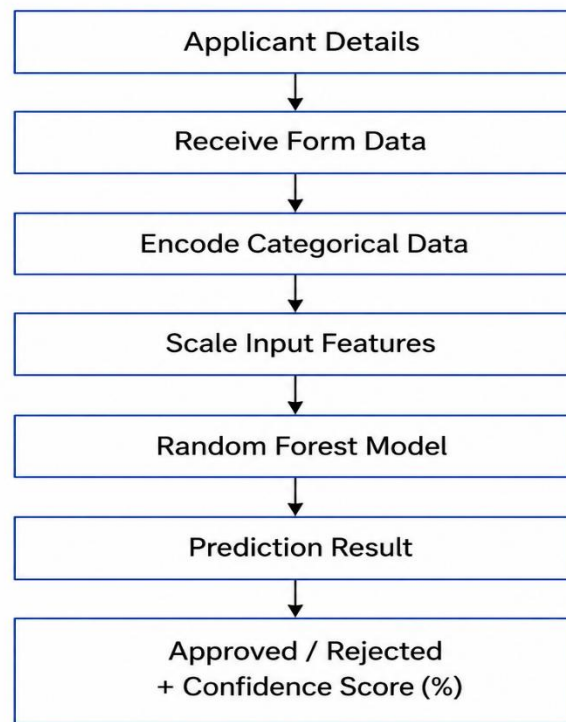
Activity 3.1: Develop the Flask Backend



In this activity, the Flask framework is used to build the backend of the application. Routes such as the home page (/) and prediction page (/predict) are created to handle user requests. The backend loads the saved machine learning model, scaler, and label encoders at startup, ensuring that the application is ready to process prediction requests efficiently.

Activity 3.2: Process Input and Generate Prediction

In this activity, the Flask backend collects applicant details from the web form, encodes categorical values, scales the numerical features, and sends the processed data to the trained Random Forest model. The model predicts whether the application is **Approved** or **Rejected**, calculates the confidence score, and displays the final result on the result page.



This milestone focuses on building the server-side logic that powers the Credit Card Approval Prediction system. The backend serves as the central component that connects the user interface with the trained machine learning model. It manages the complete flow of information by receiving user inputs, processing the data, invoking the prediction model, and delivering the final outcome to the user. The Flask backend ensures smooth communication between different components of the application, enabling fast, reliable, and real-time prediction while maintaining an organized and efficient application workflow.

Milestone 4: Frontend Development

This milestone focuses on developing an interactive and user-friendly web interface for the Credit Card Approval Prediction system. The frontend is designed using HTML and CSS to provide a clean, responsive, and easy-to-use interface where users can enter applicant details. It also displays the prediction results dynamically, ensuring users receive clear and instant feedback after submitting their information. The frontend enhances the overall user experience by providing smooth navigation and an organized layout.

Activity 4.1: Design a Responsive User Interface

In this activity, the user interface is developed using **HTML** and **CSS** to collect all the required applicant details for prediction. Input fields, dropdown menus, and the prediction button are arranged in a structured layout to improve usability. The interface is designed to be responsive so that it can be accessed comfortably on different screen sizes.

127.0.0.1:5000

Debt

3

Married (1=Yes, 0=No)

Yes

Bank Customer (1=Yes, 0=No)

Yes

Industry

CommunicationServices

Ethnicity

Asian

Years Employed

5

Prior Default (1=Yes, 0=No)

Yes

Employed (1=Yes, 0=No)

Yes

Credit Score

19

Drivers License (1=Yes, 0=No)

Yes

Citizen

ByBirth

Zip Code

500001

Income

120000

Predict Approval

127.0.0.1:5000

Debt

2

Married (1=Yes, 0=No)

Yes

Bank Customer (1=Yes, 0=No)

Yes

Industry

CommunicationServices

Ethnicity

Asian

Years Employed

4

Prior Default (1=Yes, 0=No)

No

Employed (1=Yes, 0=No)

Yes

Credit Score

5

Drivers License (1=Yes, 0=No)

Yes

Citizen

ByBirth

Zip Code

500453

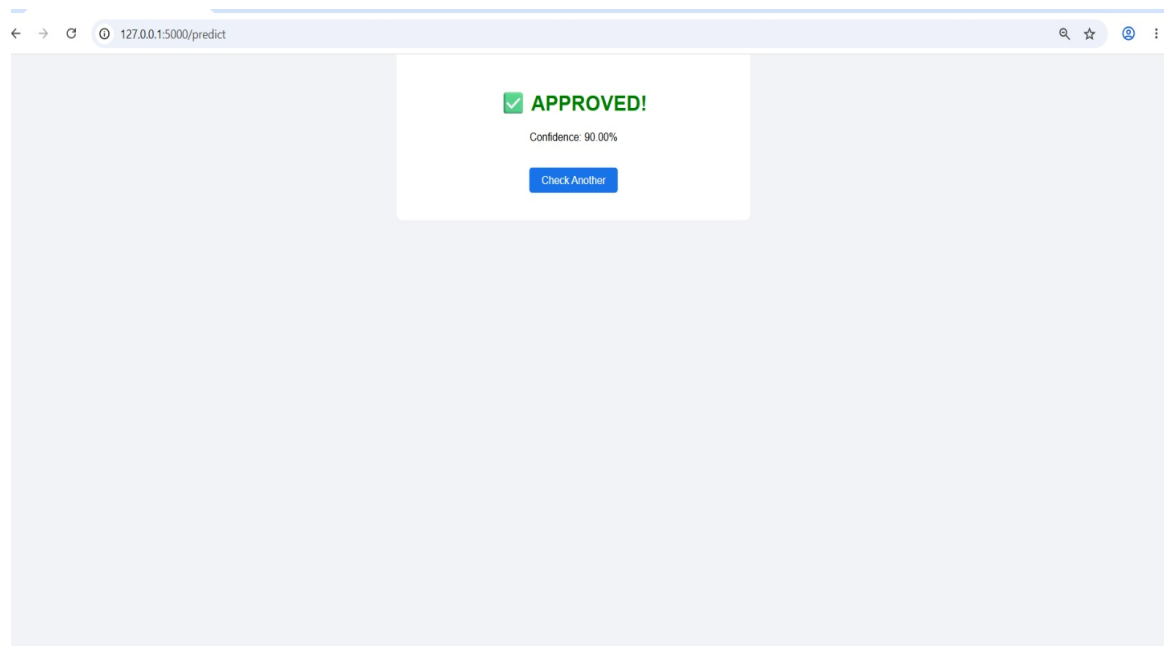
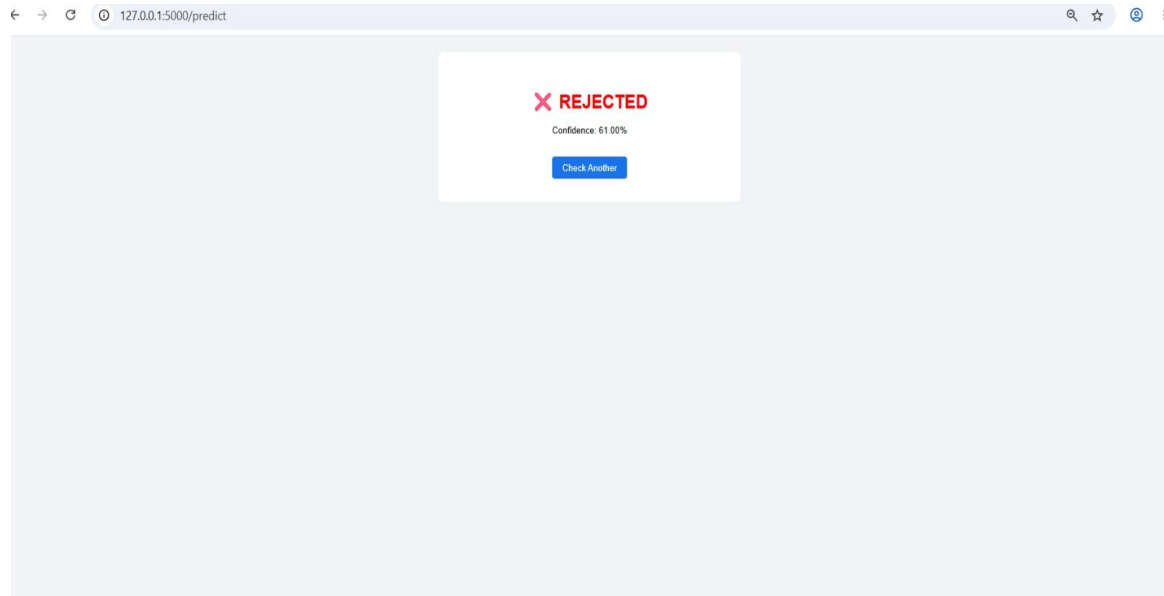
Income

80000

Predict Approval

Activity 4.2: Display Prediction Results

In this activity, Flask's `render_template()` function is used to display the prediction result page dynamically. Based on the machine learning model's prediction, the application shows whether the credit card application is Approved or Rejected, along with the confidence percentage. The result page also provides an option to return and perform another prediction.

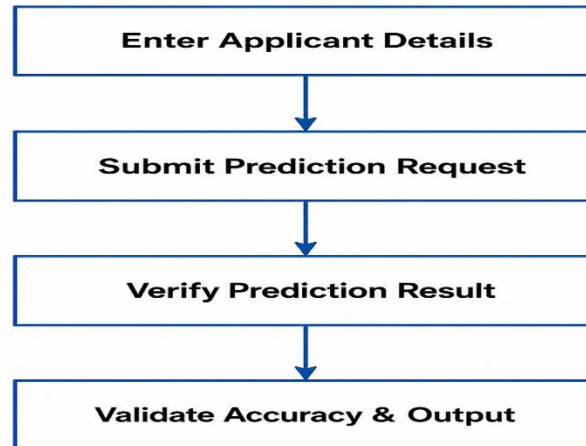


Milestone 5: Testing and Deployment

This milestone focuses on validating the functionality and reliability of the Credit Card Approval Prediction system through comprehensive testing and successful deployment on the local development environment. Different applicant data is tested to verify that the application produces accurate prediction results and operates without errors. The Flask application is then executed on the local server after installing all the required dependencies, ensuring the system is ready for demonstration and future deployment.

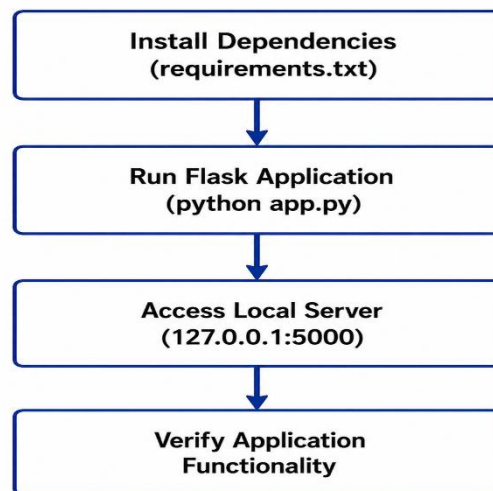
Activity 5.1: Test the Application

In this activity, the Credit Card Approval Prediction application is tested using different applicant details to verify the correctness of prediction results. Multiple test cases, including both approval and rejection scenarios, are executed to ensure that the machine learning model performs accurately. The application's response time, prediction consistency, and overall functionality are also verified to ensure reliable system performance.

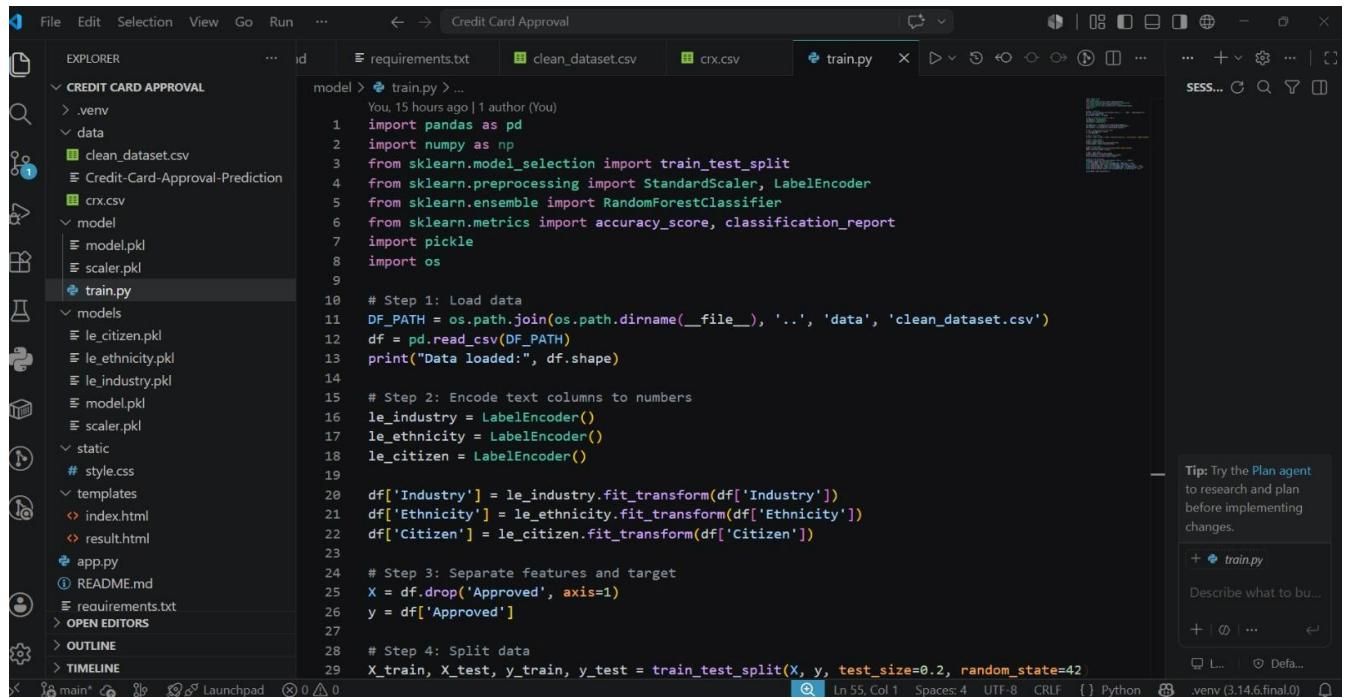


Activity 5.2: Run the Application on Flask Server

In this activity, the required Python libraries are installed using the **requirements.txt** file, and the Flask application is executed on the local development server. After the server starts successfully, the application is accessed through the browser using the local host URL. The functionality of all webpages, including the input form and prediction result pages, is verified to ensure that the application runs smoothly without any dependency or runtime errors.

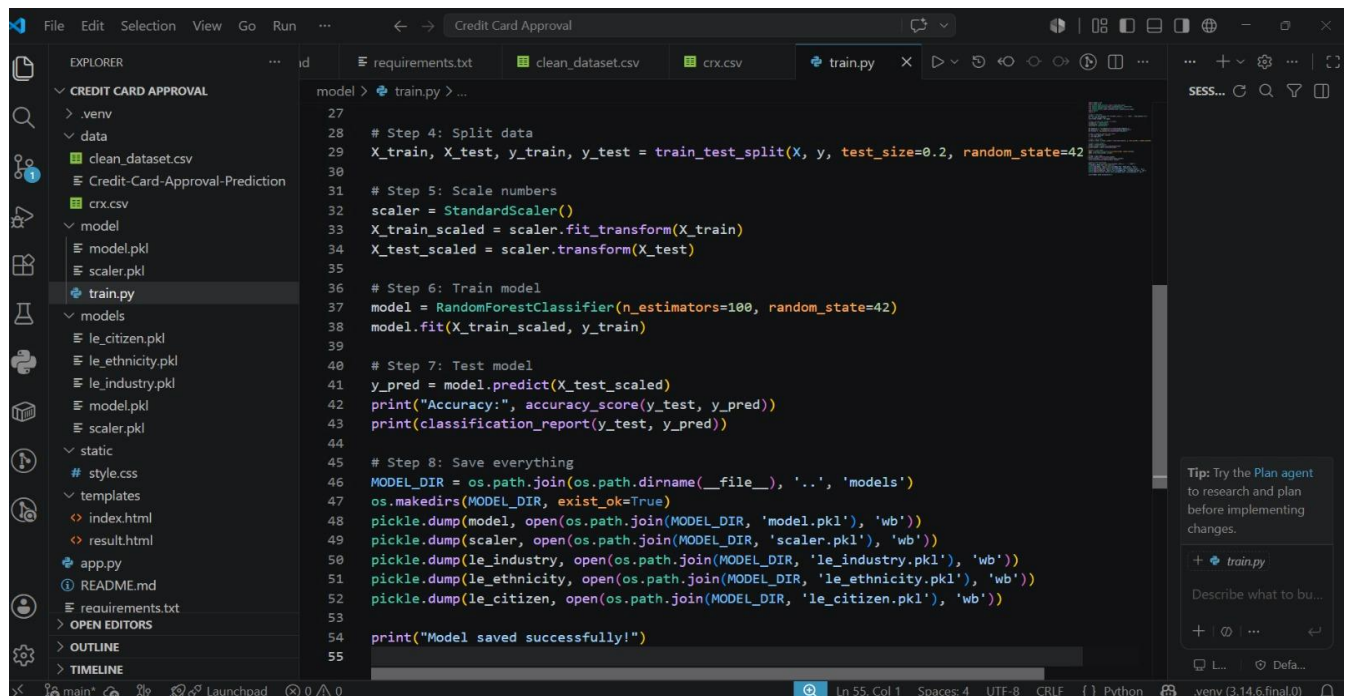


This milestone ensures that the Credit Card Approval Prediction system is stable, reliable, and ready for practical use. After completing the development of the machine learning model and web application, the entire system is thoroughly validated to confirm that all components work together correctly. Various applicant records are used to verify prediction consistency, while the frontend, backend, and trained model are tested as an integrated application. Finally, the project is executed on the local Flask development server with all required dependencies installed, confirming that users can access the application, submit applicant details, and receive accurate approval or rejection predictions with confidence scores. Successful completion of this milestone demonstrates that the application is fully functional and prepared for future enhancements or cloud deployment.



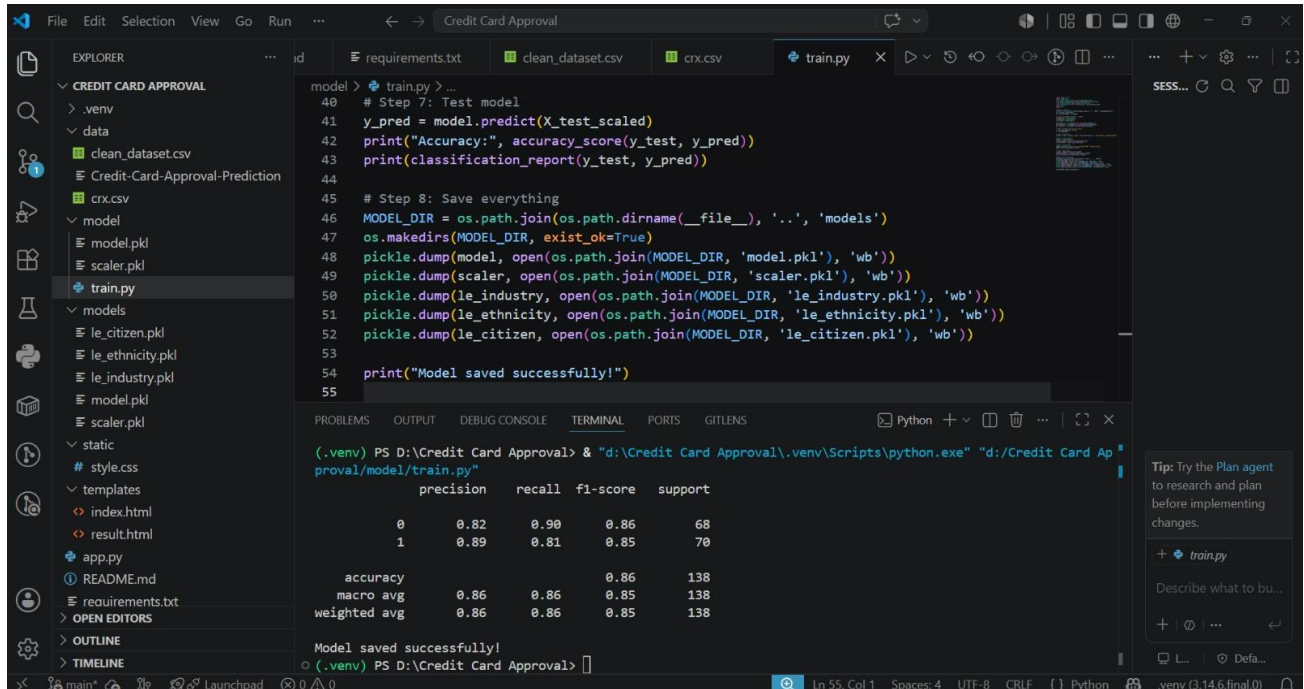
```

model > train.py > ...
You, 15 hours ago | 1 author (You)
1 import pandas as pd
2 import numpy as np
3 from sklearn.model_selection import train_test_split
4 from sklearn.preprocessing import StandardScaler, LabelEncoder
5 from sklearn.ensemble import RandomForestClassifier
6 from sklearn.metrics import accuracy_score, classification_report
7 import pickle
8 import os
9
10 # Step 1: Load data
11 DF_PATH = os.path.join(os.path.dirname(__file__), '..', 'data', 'clean_dataset.csv')
12 df = pd.read_csv(DF_PATH)
13 print("Data loaded:", df.shape)
14
15 # Step 2: Encode text columns to numbers
16 le_industry = LabelEncoder()
17 le_ethnicity = LabelEncoder()
18 le_citizen = LabelEncoder()
19
20 df['Industry'] = le_industry.fit_transform(df['Industry'])
21 df['Ethnicity'] = le_ethnicity.fit_transform(df['Ethnicity'])
22 df['Citizen'] = le_citizen.fit_transform(df['Citizen'])
23
24 # Step 3: Separate features and target
25 X = df.drop('Approved', axis=1)
26 y = df['Approved']
27
28 # Step 4: Split data
29 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
  
```



```

model > train.py > ...
27
28 # Step 4: Split data
29 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
30
31 # Step 5: Scale numbers
32 scaler = StandardScaler()
33 X_train_scaled = scaler.fit_transform(X_train)
34 X_test_scaled = scaler.transform(X_test)
35
36 # Step 6: Train model
37 model = RandomForestClassifier(n_estimators=100, random_state=42)
38 model.fit(X_train_scaled, y_train)
39
40 # Step 7: Test model
41 y_pred = model.predict(X_test_scaled)
42 print("Accuracy:", accuracy_score(y_test, y_pred))
43 print(classification_report(y_test, y_pred))
44
45 # Step 8: Save everything
46 MODEL_DIR = os.path.join(os.path.dirname(__file__), '..', 'models')
47 os.makedirs(MODEL_DIR, exist_ok=True)
48 pickle.dump(model, open(os.path.join(MODEL_DIR, 'model.pkl'), 'wb'))
49 pickle.dump(scaler, open(os.path.join(MODEL_DIR, 'scaler.pkl'), 'wb'))
50 pickle.dump(le_industry, open(os.path.join(MODEL_DIR, 'le_industry.pkl'), 'wb'))
51 pickle.dump(le_ethnicity, open(os.path.join(MODEL_DIR, 'le_ethnicity.pkl'), 'wb'))
52 pickle.dump(le_citizen, open(os.path.join(MODEL_DIR, 'le_citizen.pkl'), 'wb'))
53
54 print("Model saved successfully!")
55
  
```

The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure for 'CREDIT CARD APPROVAL'. The main editor shows the `train.py` file, which contains the following code:

```

40 # Step 7: Test model
41 y_pred = model.predict(X_test_scaled)
42 print("Accuracy:", accuracy_score(y_test, y_pred))
43 print(classification_report(y_test, y_pred))
44
45 # Step 8: Save everything
46 MODEL_DIR = os.path.join(os.path.dirname(__file__), '..', 'models')
47 os.makedirs(MODEL_DIR, exist_ok=True)
48 pickle.dump(model, open(os.path.join(MODEL_DIR, 'model.pkl'), 'wb'))
49 pickle.dump(scaler, open(os.path.join(MODEL_DIR, 'scaler.pkl'), 'wb'))
50 pickle.dump(le_industry, open(os.path.join(MODEL_DIR, 'le_industry.pkl'), 'wb'))
51 pickle.dump(le_ethnicity, open(os.path.join(MODEL_DIR, 'le_ethnicity.pkl'), 'wb'))
52 pickle.dump(le_citizen, open(os.path.join(MODEL_DIR, 'le_citizen.pkl'), 'wb'))
53
54 print("Model saved successfully!")
55

```

The terminal output shows the execution of the script, displaying a confusion matrix and model performance metrics:

```

(.venv) PS D:\Credit Card Approval> & "d:\Credit Card Approval\.venv\Scripts\python.exe" "d:\Credit Card Ap
proval\model/train.py"

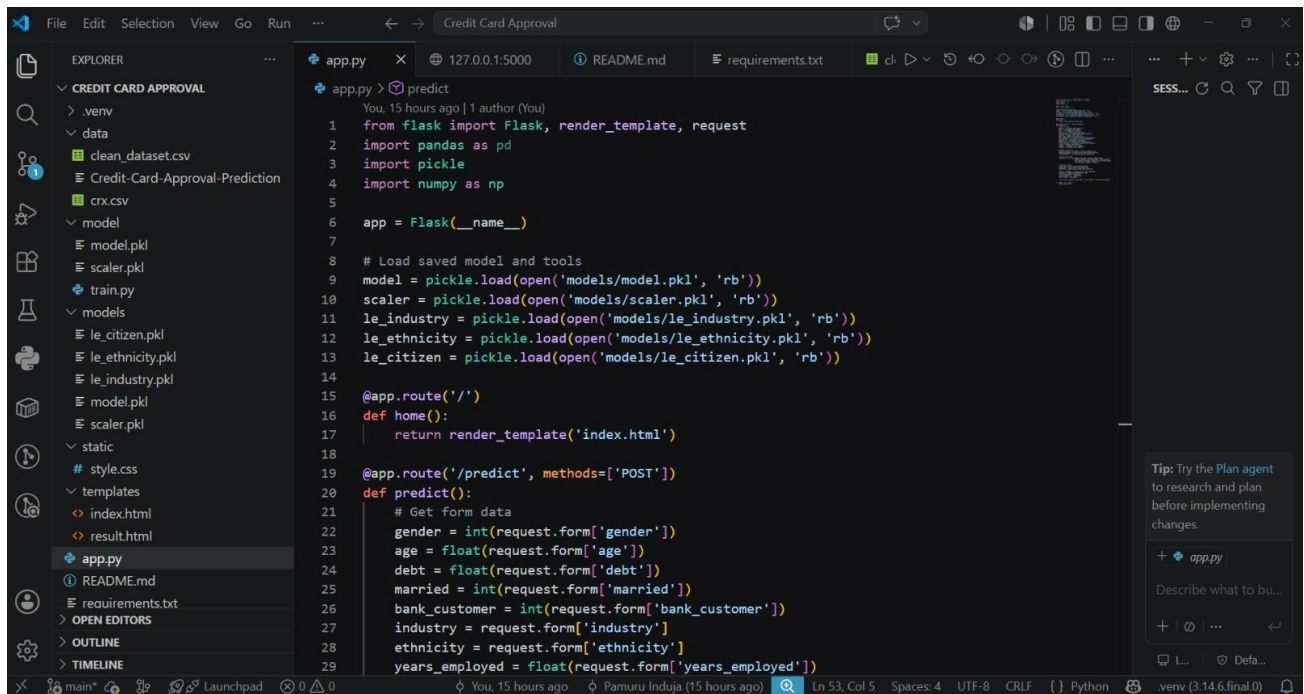
      precision    recall  f1-score   support

      0       0.82      0.90      0.86         68
      1       0.89      0.81      0.85         70

   accuracy          0.86      0.86      0.85      138
  macro avg          0.86      0.86      0.85      138
 weighted avg          0.86      0.86      0.85      138

Model saved successfully!
(.venv) PS D:\Credit Card Approval>

```

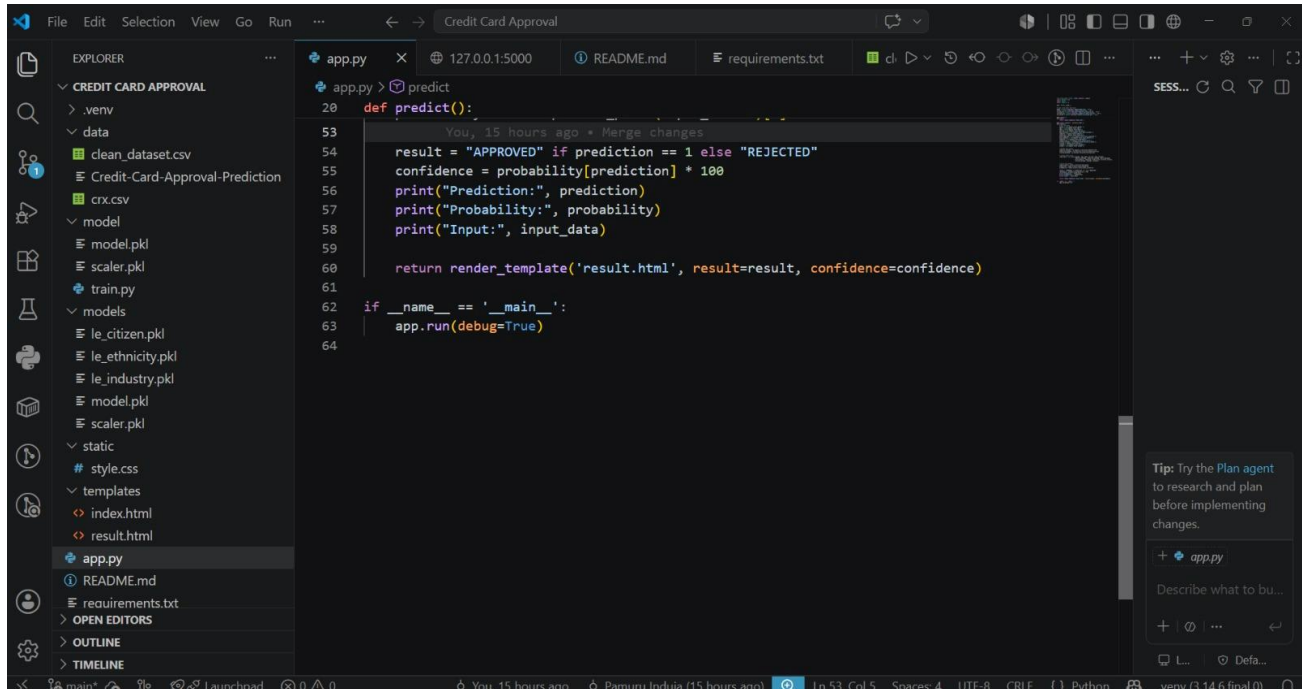


The screenshot shows the VS Code interface with the file explorer on the left displaying the project structure for 'CREDIT CARD APPROVAL'. The main editor shows the `app.py` file, which contains the following code:

```

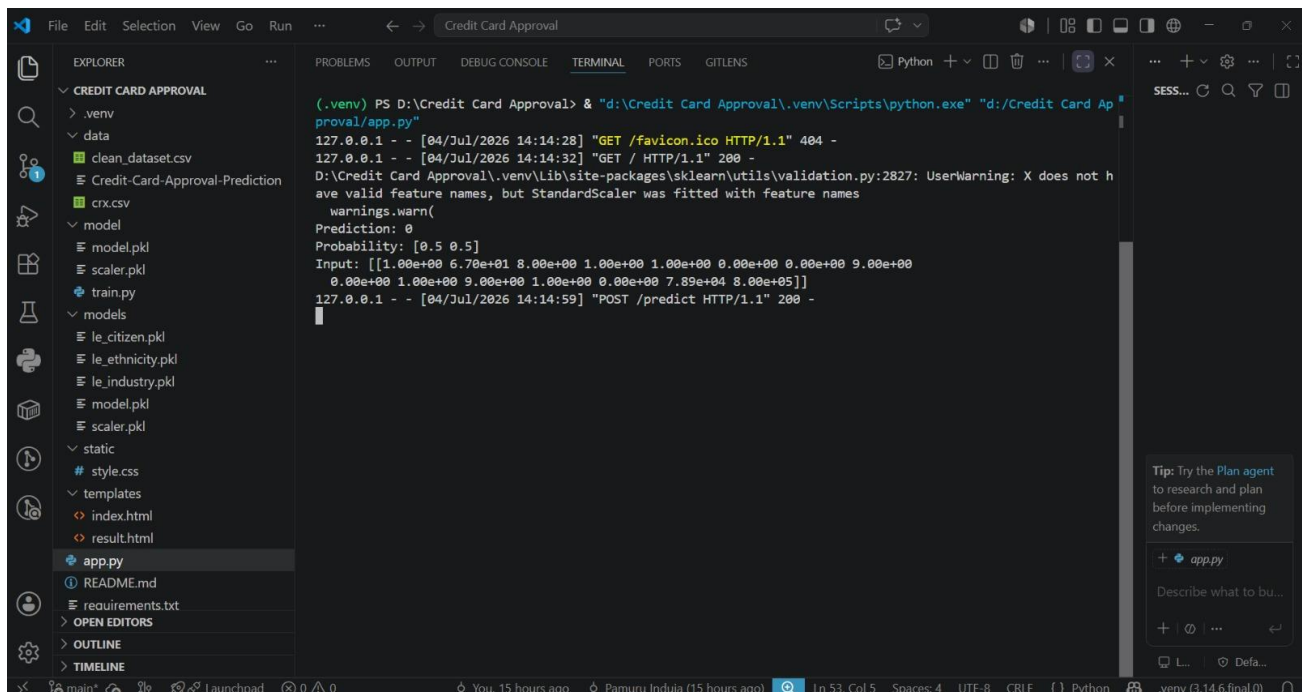
1 from flask import Flask, render_template, request
2 import pandas as pd
3 import pickle
4 import numpy as np
5
6 app = Flask(__name__)
7
8 # Load saved model and tools
9 model = pickle.load(open('models/model.pkl', 'rb'))
10 scaler = pickle.load(open('models/scaler.pkl', 'rb'))
11 le_industry = pickle.load(open('models/le_industry.pkl', 'rb'))
12 le_ethnicity = pickle.load(open('models/le_ethnicity.pkl', 'rb'))
13 le_citizen = pickle.load(open('models/le_citizen.pkl', 'rb'))
14
15 @app.route('/')
16 def home():
17     return render_template('index.html')
18
19 @app.route('/predict', methods=['POST'])
20 def predict():
21     # Get form data
22     gender = int(request.form['gender'])
23     age = float(request.form['age'])
24     debt = float(request.form['debt'])
25     married = int(request.form['married'])
26     bank_customer = int(request.form['bank_customer'])
27     industry = request.form['industry']
28     ethnicity = request.form['ethnicity']
29     years_employed = float(request.form['years_employed'])

```



```

app.py
20 def predict():
53     You, 15 hours ago * Merge changes
54     result = "APPROVED" if prediction == 1 else "REJECTED"
55     confidence = probability[prediction] * 100
56     print("Prediction:", prediction)
57     print("Probability:", probability)
58     print("Input:", input_data)
59
60     return render_template('result.html', result=result, confidence=confidence)
61
62 if __name__ == '__main__':
63     app.run(debug=True)
64
  
```



```

(.venv) PS D:\Credit Card Approval> "d:\Credit Card Approval\.venv\Scripts\python.exe" "d:/Credit Card Ap
proval/app.py"
127.0.0.1 - - [04/Jul/2026 14:14:28] "GET /favicon.ico HTTP/1.1" 404 -
127.0.0.1 - - [04/Jul/2026 14:14:32] "GET / HTTP/1.1" 200 -
D:\Credit Card Approval\.venv\Lib\site-packages\sklearn\utils\validation.py:2827: UserWarning: X does not h
ave valid feature names, but StandardScaler was fitted with feature names
warnings.warn(
Prediction: 0
Probability: [0.5 0.5]
Input: [[1.00e+00 6.70e+01 8.00e+00 1.00e+00 1.00e+00 0.00e+00 0.00e+00 9.00e+00
0.00e+00 1.00e+00 9.00e+00 1.00e+00 0.00e+00 7.89e+04 8.00e+05]]
127.0.0.1 - - [04/Jul/2026 14:14:59] "POST /predict HTTP/1.1" 200 -
  
```

Milestone 6: Conclusion

The **Credit Card Approval Prediction System** was successfully designed and developed using **Python**, the **Flask** web framework, and the **Random Forest Classifier** machine learning algorithm. The project effectively demonstrates the integration of machine learning with web application development to automate the credit card approval process. By utilizing a preprocessed dataset, feature scaling with **StandardScaler**, and label encoding for categorical attributes, the system was able to accurately learn the patterns associated with approved and rejected credit card applications. The trained model was then integrated into a Flask-based

web application, enabling users to interact with the prediction system through a simple and intuitive interface.

The application allows users to enter applicant details such as age, income, debt, employment status, credit score, marital status, and other relevant information through an easy-to-use web form. After receiving the input, the system performs the necessary preprocessing steps, including categorical value encoding and feature normalization, before passing the data to the trained Random Forest model. The model predicts whether the applicant is **Approved** or **Rejected** for a credit card and displays the prediction along with the corresponding confidence score. This real-time prediction capability makes the application efficient, reliable, and suitable for demonstrating the practical implementation of machine learning in financial decision-making.

Throughout the development process, different stages such as data preprocessing, model training, model evaluation, backend development, frontend design, testing, and local deployment were successfully completed. Extensive testing with various applicant records confirmed that the application produces consistent and accurate prediction results while maintaining smooth performance. The modular project structure, use of serialized model files through the **Pickle** library, and organized Flask application architecture make the system easy to maintain, extend, and enhance in the future.

Overall, the project achieves its primary objective of developing an intelligent **Credit Card Approval Prediction System** that simplifies the credit approval process by providing quick and data-driven predictions. The system reduces manual effort, improves decision-making efficiency, and demonstrates the practical application of machine learning in the banking and financial sector. Future improvements may include deploying the application on a cloud platform, integrating a secure database for applicant records, implementing user authentication, supporting multiple machine learning algorithms for comparison, and enhancing the user interface to provide a more interactive and responsive experience.